InstructBLIP(2023, Salesforce Research)

> InstructBLIP：图像上的指令微调（2023.05, Salesforce Research）
> InstructBLIP: Towards General-purpose Vision-Language Models with Instruction Tuning

InstructBLIP 是 BLIP 作者团队在多模态领域的又一续作，基于 **BLIP2** 提出了一个**视觉-语言的多模态指令微调系统**，加强了图文对话的能力，并提供系统性的研究来提高模型对未见过的数据和任务的泛化能力，通过统一的自然语言使通用模型能够解决各种视觉语言任务。并且作者经过实验验证，它具有 SOTA 的 zero-shot 能力。

InstructBLIP 模型**更擅长「看」、「推理」和「说」**，能够对复杂图像进行理解、推理、描述，还支持多轮对话等任务，甚至在图片解释和推理上表现优于 GPT4（一些人认为是因为基于比较强大的 BLIP2，才可以「看」的更好）。

1. 背景

在 NLP 领域，指令微调技术 (Instruction-Tuning) 帮助 LLM 推广到各种各样的任务上面。

> 做法就是对预训练好的大语言模型进行有监督微调，只是这个微调的数据集中包含了很多自然语言指令，指令微调技术使得模型能够处理并遵循人类的指令。

而且通过指令微调得到的 LLM 已经在多模态模型里面使用了，比如 BLIP-2 中的 LLM 就是。但是这些都是 LLM 领域的指令微调，只被证明了在 NLP 领域的泛化性能不错。

由于额外的视觉输入带来了丰富的输入分布和任务多样性，在构建通用的视觉-语言模型上仍具有挑战性。

大多数先前的工作大概可以分为下面两种：

✨ 多任务学习，将各种视觉语言任务制定相同的输入输出格式。
缺点：没有经过指令学习的多模态模型泛化能力不佳

✨ 用额外的视觉组件来扩展预训练 LLM，并使用 image caption 进行训练
缺点：受限于数据，无法广泛的泛化到需要超出视觉描述的多模态任务

因此，为了得到一个通用的、统一的视觉语言预训练的多模态模型，作者提出了 InstructBLIP：研究如何在视觉语言任务上进行指令微调，使得训练出的模型可以解决一系列的视觉任务。

🐞 **核心点**：

- **数据集**：
  - 将 26 个数据集转换为指令微调格式，并将它们归类到 11 个任务类别上。使用其中 13 个数据集（held-in）进行指令微调，并使用 13 个数据集（held-out）和四个完整的任务类别进行 zero-shot 评估。
  - 数据集配比：提出了一种**平衡采样策略**，以同步不同数据集间的学习进度
- **模型结构**：
  提出了一种新的指令感知视觉特征提取机制，能够**根据输入文本，提取特定的图像特征。**简单来说，**就是文本不仅输入到 LLM，也输入到 Q-Former**（以此能从冻结的图像编码器中提取指令感知的视觉特征），**Q-Former 的输出再又给到 LLM。**
- **使用的预训练模型**：
  评估和开源了一套 InstructBLIP 模型，它用了两个 LLMs 家族：
  - FlanT5：一个基于 Encoder-Decoder Transformer T5 的指令微调 LLM 模型；
  - Vicuna：一个基于 Decoder LLaMa 的微调 LLM 模型。

  InstructBLIP 在广泛的视觉语言任务上实现了最先进的 zero-shot 性能。此外，当 InstructBLIP 模型用作单个下游任务的模型初始化时，可以得到最先进的微调性能。

为了应对上述挑战，本文提出了一个视觉语言指令调整框架—InstructBLIP，使通用模型能够通过统一的自然语言接口解决各种视觉语言任务。InstructBLIP 使用不同的指令数据集来训练多模态 LLM。具体来说，使用预先训练好的 BLIP-2 模型进行初始化训练，该模型由一个图像编码器、一个 LLM 和一个查询转换器（Q-Former）组成，是二者的桥梁。在指令调整过程中，保持图像编码器和 LLM 不变，对 Q-Former 进行微调。

2. 模型方法

2.1 数据集

2.1.1 数据集划分

为了确保指令微调数据的多样性，作者收集了来自 11 种不同任务的 26 个数据集，并将它们转换为指令调优格式：

> 其中包含：图像描述、图像描述与阅读理解、视觉推理、图文问答、基于知识的图像问答、带有阅读理解的图文问答、图像问题生成（与 QA 问题相反）、视频问答、视觉对话问答、图像分类、LLaVA-Instruct-150K

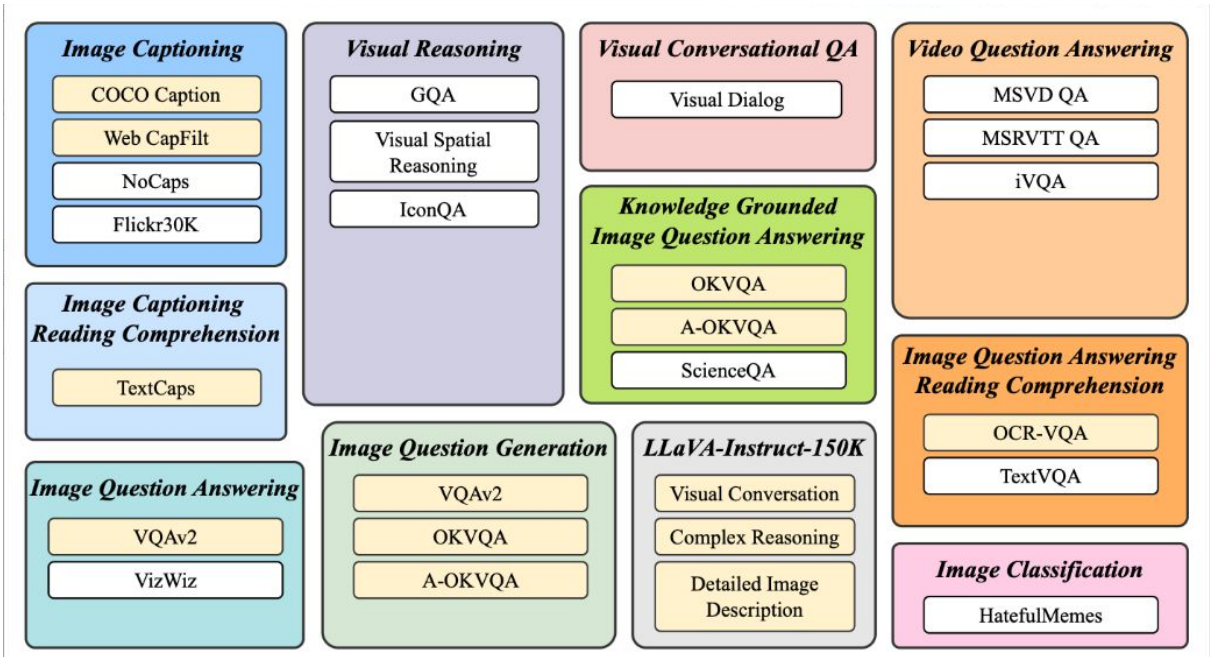


Figure 2: Tasks and their corresponding datasets used for vision-language instruction tuning. The held-in datasets are indicated by yellow and the held-out datasets by white.

训练集划分：上图中 26 个数据集中，13 个数据集（黄色，held-in data）用于 **Q-Former 微调训练**，另外 13 个数据集（白色，held-out data）用于 **zero-shot 能力测试。**

> **Held-in Data**：用于训练或调优模型的数据。这部分数据是“包含在”模型训练过程中的，可能用于训练（training set）或验证（validation set）。
> **Held-out Data**：被“保留在外”的数据，不参与模型训练，而是用于最终评估模型的泛化能力，通常是测试集（test set）。有时，在超参数调整时，特定的一部分数据也可能被作为 held-out validation set，以防止数据泄漏。

作者的目标是探究指令微调如何提高模型在没见过的数据集上的 zero-shot 表现，所以在 zero-shot 测试集中（held-out data）又继续分成两种：

1. 数据集在训练过程中没有被模型看到过，但同任务类型在训练集中出现，被模型学过，比如视觉问答任务中的 VizWiz 数据集。

> 这种类型的 held-out 数据评估并不简单，因为 held-out 数据集和训练数据集之间存在数据分布的偏移。

2. 数据不仅在训练期间没有被模型看过，而且同类型的任务也没被模型学过，比如视觉推理、视频问答、视觉对话问答和图像分类任务的数据集。

在训练期间，把所有的 held-in 数据集的训练集混合之后，并在不同任务中均匀采样 instruction templates，使用语言建模损失函数训练模型，即给模型输入图片和 instruction templates，希望它输出回答。

2.1.2 指令模版

数据集需要转化为图文指令微调的形式，用于多模态大语言模型的训练。

> 🌰：在图像分类任务中，图片 A 的类别是狗，数据的组织形式要转换成：
> 问题：图片 A，请问图片的类别是什么
> 回答：类别是狗

针对不同的任务类型，有多样化模板（10~15 个自然语言指令模板）来进行数据的形式转换：

Task	Instruction Template
Image Captioning	<Image>A short image caption: <Image>A short image description: <Image>A photo of <Image>An image that shows <Image>Write a short description for the image. <Image>Write a description for the photo. <Image>Provide a description of what is presented in the photo. <Image>Briefly describe the content of the image. <Image>Can you briefly explain what you see in the image? <Image>Could you use a few words to describe what you perceive in the photo? <Image>Please provide a short depiction of the picture. <Image>Using language, provide a short account of the image. <Image>Use a few words to illustrate what is happening in the picture.
VQA	<Image>{Question} <Image>Question: {Question} <Image>{Question} A short answer to the question is <Image>Q: {Question} A: <Image>Question: {Question} Short answer: <Image>Given the image, answer the following question with no more than three words. {Question} <Image>Based on the image, respond to this question with a short answer: {Question}. Answer: <Image>Use the provided image to answer the question: {Question} Provide your answer as short as possible: <Image>What is the answer to the following question? "{Question}" <Image>The question "{Question}" can be answered using the image. A short answer is
VQG	<Image>Given the image, generate a question whose answer is: {Answer}. Question: <Image>Based on the image, provide a question with the answer: {Answer}. Question: <Image>Given the visual representation, create a question for which the answer is "{Answer}". <Image>From the image provided, craft a question that leads to the reply: {Answer}. Question: <Image>Considering the picture, come up with a question where the answer is: {Answer}. <Image>Taking the image into account, generate an question that has the answer: {Answer}. Question:

Table 5: Instruction templates used for transforming held-in datasets into instruction tuning data. For datasets with OCR tokens, we simply add "OCR tokens:" after the image query embeddings.

这些模板清晰指出任务和目标，可以分为 3 类：

- 图像描述
  - 🌰：“A shot image caption:”“A photo of ”
  - 一共 13 个模板
- 视觉问答
  - 🌰：“{直接贴问题}”、“Q:{问题} A:”
  - 一共 10 个模板
- 视觉问题生成
  - 🌰：“Geven the image，generate a question whose answer is:{Answer}. Question:”

- 一共 6 个模板

⚠

1. 对于一些偏向或容易有简短回答的数据集，作者刻意地在 instruction 中添加了`short answer`、`as short as possible`或是`briefly`字样来降低模型过拟合的风险，防止其始终生成很短的输出。（**防止过拟合的方式是：在指令中有所体现**）
2. 对于 LLaVA-Instruct-150K 数据集，由于其指令格式是自然结构的，因此无需添加额外的指令模板。
3. 对于涉及场景文本的数据集，OCR 标记作为补充信息添加到指令中。

2.1.3 均衡训练集

🤔：由于训练数据集的数量众多，而且每个数据集的大小差异很大，均匀地混合这些数据集可能会导致->模型在小数据集上过拟合，在大数据集上欠拟合。

☑：提出了一种按照数据集大小加权采样的方法。

具体采样方法：

假设有 D 个数据集，每个数据的大小为 $S_1, S_2, ..., S_D$，那么第 d 个数据集的采样概率（从该数据集中选取一个训练样本的概率）为：$p_d = \sqrt{S_d} / \sum_{i=1}^{D} \sqrt{S_i}$。

同时作者还对某些数据集的权重进行了手动微调，以提高其收敛性。特别是降低了以多选题为特色的 A-OKVQA 的权重，增加了需要生成开放式文本的 OKVQA 的权重。。

> 因：数据集和任务之间存在固有的差异，尽管数据集和任务的规模相似，但却需要不同程度的训练强度。

2.2 模型结构 & 训练策略

在当时，像 BLIP-2 这种现有的零样本图像到文本的生成方法，在提取视觉特征时采取了与指令并无关系的方法。也就是说，LLM 中的视觉输入是不知道指令的（**Instruction-agnostic**），这并不利于该模型在不同任务中的灵活性。

因此，如果模型可以适应多个任务类型，将任务类型作为指令信息之一，来产生相关的视觉表征，就能使得模型在任务类型上获得零样本能力。这样的话，相同的输入图像，在不同的**任务指令**下会有很大的变化。所以，指令感知的视觉模型可以提高模型从不同指令中学习的能力。

⛰ 考虑两种情况：（可以认为这是**指令调整的优点**）

1️⃣ 输入同一图像，模型被要求完成两个不同的任务 ➡➡一个指令感知的视觉模型可以根据指令从同一图像中提取不同的特征，在解决不同任务时呈现出更多的信息特征

2️⃣ 给定两个不同的图像，模型被指示完成同一任务 ➡➡ 一个指令感知的视觉模型可以利用指令中体现的共同知识来提取两个不同图像的特征，从而实现图像之间更好的信息转移

InstructBLIP 通过充分使用 BLIP-2 模型中的 Q-Former 架构，提出了一种指令感知的视觉特征提取方法。

InstructBLIP 的模型架构和 BLIP-2 类似，区别在于输入文本换成了指令数据 Instructions，都包含**图像编码器、Q-Former 和 LLM**，并使用预训练的 BLIP-2 模型进行初始化。

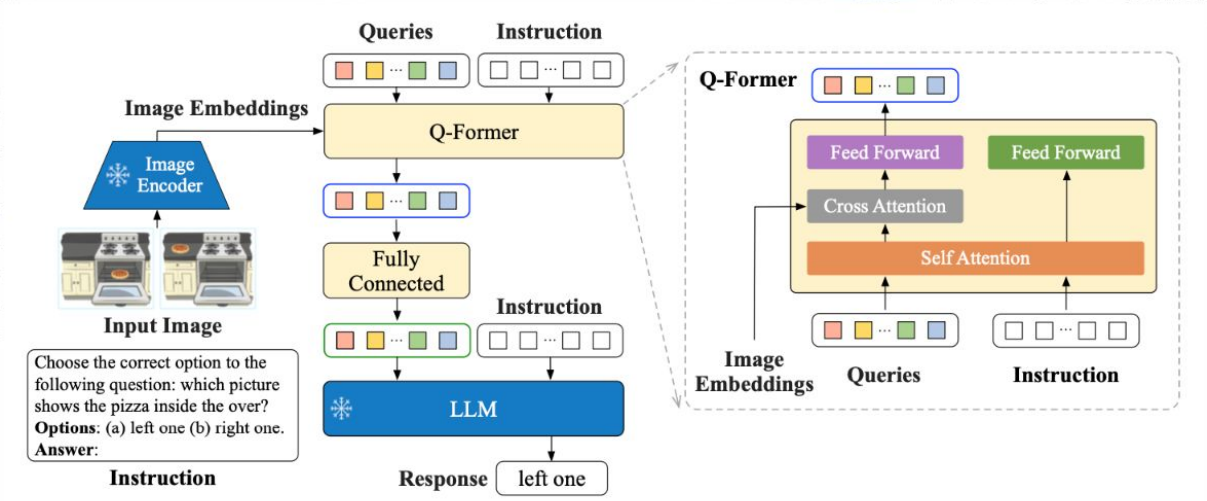


Figure 3: Model architecture of InstructBLIP. The Q-Former extracts instruction-aware visual features from the output embeddings of the frozen image encoder, and feeds the visual features as soft prompt input to the frozen LLM. We instruction-tune the model with the language modeling loss to generate the response.

模型的前向过程如下所示：

1. 输入的图片通过冻结的图像编码器提取图像特征，并输入到 Q-Former 中，此外 Q-Former 的输入还包括可学习的 Queries (BLIP-2 的做法) 和 Instruction。

   > Q-Former 的输入：图像特征 + 可学习的 Queries + Instruction

2. Q-Former 的内部结构如上图右侧所示，可学习的 Queries 通过 **Self-Attention** 和 Instruction 交互，然后通过 **Cross-Attention** 和输入图片的特征交互，用于提取与 Instruction 相关的图像特征。
3. Q-Former 的输出通过一个 FC 进行 transform 后送入到冻结的 LLM 中，生成回复。

Q-Former 在指令微调前的预训练过程遵循 BLIP-2 的两步：

1. 预训练 1：首先冻结图像编码器的参数，预训练 Q-Former 的参数，训练目标是视觉语言建模 (ITC、MLM、ITM)。使用 image caption 数据，学习视觉文本相关性表示。
2. 预训练 2：其次冻结 LLM 的参数，预训练 Q-Former 的参数，训练目标是文本生成（LM）。使用 image caption 数据，学习对齐 LLM 的文本生成。
3. 指令微调：在预训练之后，通过指令调整对 Q-Former 进行微调，其中 LLM 接收来自 Q-Former 的视觉编码和任务指令作为输入（Q-Former 的输出 + Instruction 在 embedding 层面融合，输入到 LLM 中）。使用 Instruction 任务数据，学习遵循指令生成回复的能力。

⚠ 在 InstructBLIP 中，指令文本不仅作为输入到了 LLM 中，同时也输入到了 Q-Former 中。
指令 instruction 不仅会指导 LLM 生成文本，同时也会提取不同的视觉特征

指令信息使得 Q-Former 模型更加关注图像特征中和任务相关的信息。从 prompt 的角度来看，图像特征是一种**软提示（soft prompt）**，指令是一种相对较“硬”的提示。

✸ 一句话概括：
Q-Former 提取指令感知的图像特征（Instruction-aware vision model），根据指令的不同获取不同的图像特征。然后将这些图像特征作为 LLM 的软视觉提示（soft prompt），使用 LM loss 和指令微调模型生成回复。

在 InstructBLIP 中，Q-Former 如何根据不同的指令筛选不同的视觉细节？

在 **InstructBLIP** 的架构中，Q-Former 的核心作用是作为 **视觉-语言的桥梁**，它的关键特性是**动态筛选与指令相关的视觉特征。**

1. **Q-Former 通过 Instruction 引导 Queries 的关注区域**

   在 **前向传播（Forward Pass）**中，**Q-Former 内部处理（关键部分）**：

   A. **第一步：Queries + Instruction 通过 Self-Attention 交互**

   - Queries 并不是静态的，而是可以自适应地调整它们对图像的关注点。
   - 用户的指令 **会影响 Queries 的自注意力机制（Self-Attention）**，让它们对不同任务产生不同的搜索偏好。

   B. **第二步：Queries 与图像特征通过 Cross-Attention 交互**

   - Queries 通过 Cross-Attention 从图像特征中提取 **与 Instruction 相关的视觉信息**，而不是盲目读取所有图像信息。
   - **不同的 Instruction 会改变 Queries 的关注点**：
     - “这是什么动物?” → Queries 关注整体形状和轮廓。
     - “它的眼睛是什么颜色?” → Queries 关注局部区域（眼睛）。
     - “背景里有什么?” → Queries 关注背景，而忽略主体。

2. **训练过程：确保 Q-Former 学会“选择”合适的视觉信息**

   Q-Former 的训练策略保证了：

   - Q-Former 既不会把所有图像信息都传给 LLM（避免冗余），也不会遗漏关键细节（提升任务相关性）。
   - 指令不同，Q-Former 选择的信息就不同，使模型具有高度的指令适应能力。

Q-Former 能够根据指令选择不同的视觉细节，主要依赖于：

- **Self-Attention 使 Queries 理解指令，Cross-Attention 让 Queries 选择相关图像特征。**
- **通过两阶段训练，Q-Former 学会根据不同指令自动筛选关键信息，而非盲目传递所有图像信息。**
- **LLM 仅接受经过 Q-Former 处理后的视觉特征，从而根据用户指令生成合理的回答。**

训练细节：

```
@registry.register_model("blip2_vicuna_instruct")
class Blip2VicunaInstruct(Blip2Base):
    """
    BLIP2 Vicuna 模型。
    支持的模型类型:
        - vicuna7b
        - vicuna13b
    用法示例:
        >>> from lavis.models import load_model
        >>> model = load_model("blip2_vicuna_instruct", "vicuna7b")
    """

    ......

    def __init__(
        self,
        vit_model="eva_clip_g",           # vision encoder 模型名称
        img_size=224,                     # 输入图像尺寸
        drop_path_rate=0,                 # DropPath 的比率
        use_grad_checkpoint=False,        # 是否使用梯度检查点，以节省显存
```

```
    vit_precision="fp16",            # vision encoder 的精度
    freeze_vit=True,                  # 是否冻结 vision encoder 的参数
    num_query_token=32,              # Q-former 的可学习查询 token 数量
    llm_model="",                     # LLM (Vicuna) 的检查点路径或名称
    prompt="",                        # 默认提示语字符串
    max_txt_len=128,                  # 输入文本的最大长度
    max_output_txt_len=256,           # 生成文本的最大长度
    apply_lemmatizer=False,           # 是否在推理后对文本进行词形还原
    qformer_text_input=True,          # Q-former 是否接收文本输入
):

    ......

    # 是否冻结视觉编码器
    if freeze_vit:
        for name, param in self.visual_encoder.named_parameters():
            param.requires_grad = False
        self.visual_encoder = self.visual_encoder.eval()
        self.visual_encoder.train = disabled_train  # 禁用训练
        logging.info("freeze vision encoder")

    # 初始化 Q-former 模块以及可学习的查询 token
    self.Qformer, self.query_tokens = self.init_Qformer(
        num_query_token, self.visual_encoder.num_features
    )

    # 如果不需要 Q-former 的文本输入，则把对应的层置为 None
    if not qformer_text_input:
        self.Qformer.bert.embeddings.word_embeddings = None
        self.Qformer.bert.embeddings.position_embeddings = None
        for layer in self.Qformer.bert.encoder.layer:
            layer.output = None
            layer.intermediate = None
    else:
        # 如果需要文本输入，就调整 Q-former 的词表大小以适配 tokenizer
        self.Qformer.resize_token_embeddings(len(self.tokenizer))
    self.Qformer.cls = None  # 不需要分类头

    # 初始化 Llama tokenizer
    self.llm_tokenizer = LlamaTokenizer.from_pretrained(llm_model, use_fast=False,
truncation_side="left")
    # 从预训练权重中加载 LlamaForCausalLM 模型
    self.llm_model = LlamaForCausalLM.from_pretrained(
        llm_model, torch_dtype=torch.float16
    )

    ......

    # 重新 resize LLM 的词嵌入 (embedding) 层大小，以匹配添加了新 token 的 tokenizer
    self.llm_model.resize_token_embeddings(len(self.llm_tokenizer))

    # 下面是另一种获取 eos_token_id 的方法，这里被注释了
    # self.eos_token_id = self.llm_tokenizer(
    #     self.llm_tokenizer.eos_token, add_special_tokens=False
    # ).input_ids[0]
```

```
        # 冻结 LLM 参数，使其不可训练
        for name, param in self.llm_model.named_parameters():
            param.requires_grad = False

        # 线性投影层，用于将 Q-former 输出向量映射到 LLM 的隐藏维度
        self.llm_proj = nn.Linear(
            self.Qformer.config.hidden_size, self.llm_model.config.hidden_size
        )

        ......

        self.qformer_text_input = qformer_text_input  # 是否在 Q-former 阶段允许文本输入

    def concat_text_input_output(self, input_ids, input_atts, output_ids, output_atts):
        """
        将文本输入与输出拼接到一起，供 LLM 做统一的训练（即在顺序上：输入 + 输出 + 输入剩余部分）。
        """

        ......

        return llm_tokens, input_part_targets_len

    def forward(self, samples):
        """
        前向传播，用于训练阶段（有监督训练）。
        接收 samples，其中包括图像和文本（输入/输出）等内容。
        """
        # image 是输入图像数据
        image = samples["image"]
        with self.maybe_autocast():
            # 用视觉编码器得到图像特征
            image_embeds = self.ln_vision(self.visual_encoder(image))
        # 构建图像特征的 attention mask，全是 1
        image_atts = torch.ones(image_embeds.size()[:-1], dtype=torch.long).to(image.device)

        bs = image.size(0)  # batch size

        # 将可学习查询 token 扩展成 batch 数量
        query_tokens = self.query_tokens.expand(image_embeds.shape[0], -1, -1)
        # 如果在 Q-former 中需要文本输入
        if self.qformer_text_input:
            # 利用 tokenizer 对 samples["text_input"] 进行处理
            text_Qformer = self.tokenizer(
                samples["text_input"],
                padding='longest',
                truncation=True,
                max_length=self.max_txt_len,
                return_tensors="pt",
            ).to(image.device)
            # 对查询 token 的 attention mask 也全为 1
            query_atts = torch.ones(query_tokens.size()[:-1], dtype=torch.long).to(image.device)
            # 将两部分的 atts 拼接
            Qformer_atts = torch.cat([query_atts, text_Qformer.attention_mask],dim=1)

            # 送入 Q-former 的 bert
            query_output = self.Qformer.bert(
```

```
            text_Qformer.input_ids,
            attention_mask=Qformer_atts,
            query_embeds=query_tokens,
            encoder_hidden_states=image_embeds,
            encoder_attention_mask=image_atts,
            return_dict=True,
        )
    else:
        # 不带文本输入的 Q-former
        query_output = self.Qformer.bert(
            query_embeds=query_tokens,
            encoder_hidden_states=image_embeds,
            encoder_attention_mask=image_atts,
            return_dict=True,
        )

    # 只取查询 token 对应的输出部分，并通过线性层映射到 LLM 的隐藏维度
    inputs_llm = self.llm_proj(query_output.last_hidden_state[:,:query_tokens.size(1),:])
    # 对应的 attention 全部为 1
    atts_llm = torch.ones(inputs_llm.size()[:-1], dtype=torch.long).to(image.device)

    # LLM 的 tokenizer；生成输入 tokens
    self.llm_tokenizer.padding_side = "right"
    self.llm_tokenizer.truncation_side = 'left'
    text_input_tokens = self.llm_tokenizer(
        samples['text_input'],
        return_tensors="pt",
        padding="longest",
        truncation=True,
        max_length=self.max_txt_len,
    ).to(image.device)

    # 输出文本 tokens（也就是标签）
    self.llm_tokenizer.truncation_side = 'right'
    text_output_tokens = self.llm_tokenizer(
        [t + self.llm_tokenizer.eos_token for t in samples['text_output']],
        return_tensors="pt",
        padding="longest",
        truncation=True,
        max_length=self.max_output_txt_len,
    ).to(image.device)

    # 将输入与输出拼接，以得到真正送入 LLM 的内容
    llm_tokens, input_part_targets_len = self.concat_text_input_output(
        text_input_tokens.input_ids,
        text_input_tokens.attention_mask,
        text_output_tokens.input_ids,
        text_output_tokens.attention_mask,
    )

    ......

    # 将 LLM embedding 与 tokens 拼接
    inputs_embeds = self.llm_model.get_input_embeddings()(llm_tokens['input_ids'])
    inputs_embeds = torch.cat([inputs_llm, inputs_embeds], dim=1)
    # 同步拼接 attention mask
```

```
        attention_mask = torch.cat([atts_llm, llm_tokens['attention_mask']], dim=1)

        # 最后传给 LLM 做因果语言建模训练
        with self.maybe_autocast():
            outputs = self.llm_model(
                inputs_embeds=inputs_embeds,
                attention_mask=attention_mask,
                return_dict=True,
                labels=targets,
            )

        loss = outputs.loss  # 计算出的 loss

        return {"loss": loss}  # 返回一个字典格式，包含 loss

    @torch.no_grad()
    def generate(
        self,
        samples,
        use_nucleus_sampling=False,  # 是否使用核采样
        num_beams=5,                 # beam search 的大小
        max_length=256,              # 生成的最大长度
        min_length=1,                # 生成的最小长度
        top_p=0.9,                   # nucleus sampling 的阈值
        repetition_penalty=1.5,      # 重复惩罚系数
        length_penalty=1,            # 长度惩罚系数
        num_captions=1,              # 生成的序列个数
        temperature=1,               # 温度系数
    ):
        """
        推理模式下的文本生成函数。
        """

        self.llm_tokenizer.padding_side = "left"

        # 如果 samples 中包含 "prompt"，就用它，否则用初始化时给定的 self.prompt
        if "prompt" in samples.keys():
            prompt = samples["prompt"]
        else:
            prompt = self.prompt

        ......

        # 如果有 OCR tokens，且 prompt 中包含 "{}"，就把 OCR tokens 替换进去（针对某些文本任务）
        if "ocr_tokens" in samples.keys() and "{}" in prompt[0]:
            prompt = [p.format(', '.join(samples['ocr_tokens'][i][:30])) for i, p in enumerate(
    prompt)]

        ......

        # 针对视频输入（5 维张量，形状 [B, C, T, H, W]）
        if image.dim() == 5:
            ......
        else:
            # 普通情况（单帧图像）
            ......
```

```
        # 对 prompt 做 LLM tokenizer
        llm_tokens = self.llm_tokenizer(
            prompt,
            padding="longest",
            return_tensors="pt"
        ).to(image.device)

        # 在自动混合精度下，获取 LLM 的输入嵌入，并与图像查询部分拼起来
        with self.maybe_autocast():
            inputs_embeds = self.llm_model.get_input_embeddings()(llm_tokens.input_ids)
            inputs_embeds = torch.cat([inputs_llm, inputs_embeds], dim=1)
            attention_mask = torch.cat([atts_llm, llm_tokens.attention_mask], dim=1)

            # 使用 HuggingFace 的 generate 函数进行文本生成
            outputs = self.llm_model.generate(
               ......
            )

        ......

        return output_text
```

作者重写了 bert embedding 层的代码，将 query_embeds（可理解为 Q-Former 的输出）和 embeddings（可理解为 Instruction 的文本 embedding）concat 起来

3. 推理任务

对于不同的任务类别，采用不同的推理策略。

⏏：

- 对于大部分描述性任务，如 image captioning，open-ended VQA 等，InstructBLIP 可以直接使用 LLM 生成的文本作为输出（采用传统的 transformer 解码方式生成回答）。
- 对于选择性任务，如 classification 和 multi-choice VQA 等这种回复内容受限的任务，InstructBLIP 遵循 ALBEF 的做法生成固定的几种答案，根据概率选择最后的结果作为输出。实际上就是 vocabulary ranking method，将 LLM 生成的内容词汇限制为候选列表进行排序（将词汇量限制在候选列表中），计算每个候选的对数似然，选择数值最大的一个作为最终预测。这种排序方法适用于 ScienceQA、IconQA、A-OKVQA（多选）、HatefulMemes、Visual Dialog、MSVD 和 MSRVTT 数据集。